

OEL PROPOSAL

E-Commerce Sales Analysis

Advanced Exploratory Data Analysis Using Python

Umar Muneer (75600)

Hafiz Sharjeel Ali(75601)

Muhammad Ali Raza (62696)

Haider Hassan (68119)

My Github

<https://github.com/sharjeel6636/OEL-Ecommerce-Analysis>

1. Topic

E-Commerce Sales Analysis Using Python, NumPy, Pandas, Matplotlib & Seaborn

DataSet: <https://www.kaggle.com/datasets/carrie1/ecommerce-data?resource=download>

2. Description

This project performs a comprehensive Exploratory Data Analysis (EDA) on an e-commerce sales dataset. The analysis uncovers hidden patterns in customer behavior, product performance, regional sales trends, and revenue seasonality. By leveraging Python's powerful data science stack — NumPy, Pandas, Matplotlib, and Seaborn — the project transforms raw transactional data into actionable business insights through rich statistical summaries and visualizations.

3. Problem Statement

E-commerce businesses generate massive volumes of sales data daily, but without proper analysis, this data remains unutilized. Decision-makers struggle to identify:

- Which products drive the most revenue and which underperform
- Which customer segments are most valuable (RFM Analysis)
- What seasonal or monthly patterns affect total sales
- Which regions or cities contribute most to business growth

This project addresses these challenges by applying structured EDA techniques to extract meaningful patterns from raw sales data.

4. Objectives

- Perform full data cleaning and preprocessing using Pandas
- Conduct descriptive statistical analysis using NumPy
- Identify top-selling products and top revenue-generating countries
- Perform RFM (Recency, Frequency, Monetary) customer segmentation
- Analyze monthly and seasonal revenue trends
- Detect outliers in sales and order quantities
- Visualize all insights using Matplotlib and Seaborn

5. Methodology

5.1 Data Collection

- Dataset: Online Retail E-Commerce Dataset (Kaggle/UCI – UK-based transactional data)
- Columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country

5.2 Data Preprocessing

- Handle missing values using Pandas fillna() and dropna()
- Convert date columns to datetime format
- Remove duplicate records
- Feature engineering: extract Month, Year, Day-of-Week from order dates

5.3 Exploratory Data Analysis

- Descriptive statistics: mean, median, std, skewness using NumPy
- Correlation matrix and heatmap to find variable relationships
- Top 10 products by revenue
- Monthly revenue trend line chart
- Regional sales bar chart and pie chart

5.4 RFM Analysis

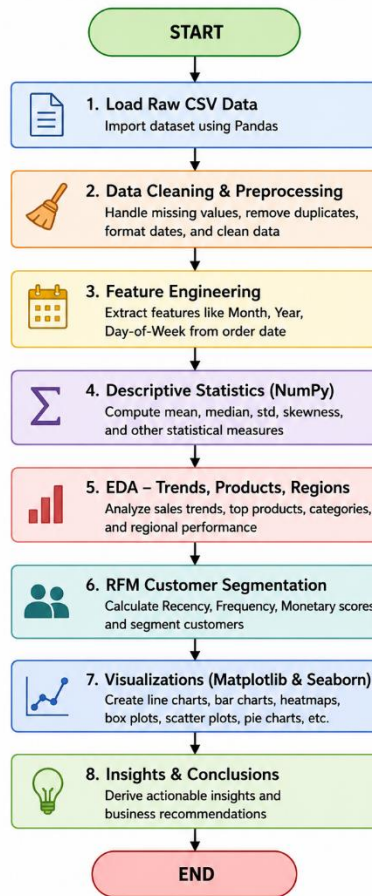
- Calculate Recency, Frequency, and Monetary scores per customer
- Segment customers into: Champions, Loyal, At-Risk, Lost
- Visualize segments using Seaborn count plots and scatter plots

5.5 Visualization

- Line charts — revenue trends over time
- Bar charts — product and category performance
- Heatmaps — correlation and seasonal patterns
- Box plots — outlier detection in sales values
- Scatter plots — price vs. profit, quantity vs. revenue
- Pie charts — regional contribution breakdown

6. Flowchart

FLOWCHART – E-COMMERCE SALES ANALYSIS



7. Tools & Techniques

Tool / Library	Version	Purpose
Python	3.11+	Core programming language
NumPy	1.26+	Numerical computations & statistics
Pandas	2.1+	Data manipulation & preprocessing
Matplotlib	3.8+	Base charting & custom plots
Seaborn	0.13+	Statistical & advanced visualizations
Jupyter Notebook	7.0+	Interactive development environment

8. Expected Outcomes

-
- A fully cleaned and preprocessed e-commerce dataset ready for analysis
 - Identification of top 10 revenue-generating products
 - Monthly and seasonal revenue trend visualizations
 - Customer segmentation report based on RFM scoring
 - Regional performance breakdown with visual comparisons
 - A comprehensive EDA report with 10+ professional visualizations
 - Actionable business insights and recommendations for the e-commerce platform

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (12, 5)

print('All libraries imported successfully!')

df = pd.read_csv('data.csv', encoding='latin1')

print('Dataset Shape:', df.shape)
print('\nFirst 5 Rows:')
df.head()

print('Column Names:', df.columns.tolist())
print('\nData Types:')
print(df.dtypes)

# — Missing Values —
print('Missing Values Before Cleaning:')
print(df.isnull().sum())

# Drop rows with missing CustomerID (required for RFM)
df.dropna(subset=['CustomerID'], inplace=True)
```

```

# Corrected fillna deployment to avoid inplace warnings
df['Description'] = df['Description'].fillna('Unknown')

print('\nMissing Values After Cleaning:')
print(df.isnull().sum())

# — Remove Duplicates —
before = df.shape[0]
df.drop_duplicates(inplace=True)
after = df.shape[0]
print(f'Duplicates removed: {before - after} rows')

# — Remove Cancelled Orders & Invalid Prices —
df = df[df['Quantity'] > 0]
df = df[df['UnitPrice'] > 0]

print('Clean dataset shape:', df.shape)

# — Feature Engineering —
# Added explicit string matching format to optimize performance speeding up parse times
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'], format='%m/%d/%Y %H:%M')

df['Revenue'] = df['Quantity'] * df['UnitPrice']
df['Month'] = df['InvoiceDate'].dt.month
df['Year'] = df['InvoiceDate'].dt.year
df['DayOfWeek'] = df['InvoiceDate'].dt.day_name()
df['CustomerID'] = df['CustomerID'].astype(int)

print('New columns added: Revenue, Month, Year, DayOfWeek')
df[['InvoiceDate', 'Revenue', 'Month', 'Year', 'DayOfWeek']].head()

revenue = df['Revenue'].values
qty = df['Quantity'].values

print('==== Revenue Statistics ====')
print(f'Mean : £{np.mean(revenue):.2f}')
print(f'Median : £{np.median(revenue):.2f}')

```

```

print(f'Std Dev : £{np.std(revenue):.2f}')
print(f'Skewness: {pd.Series(revenue).skew():.2f}')
print(f'Min    : £{np.min(revenue):.2f}')
print(f'Max    : £{np.max(revenue):.2f}')

print('\n===== Quantity Statistics =====')
print(f'Mean    : {np.mean(qty):.2f}')
print(f'Median  : {np.median(qty):.2f}')
print(f'Std Dev : {np.std(qty):.2f}')

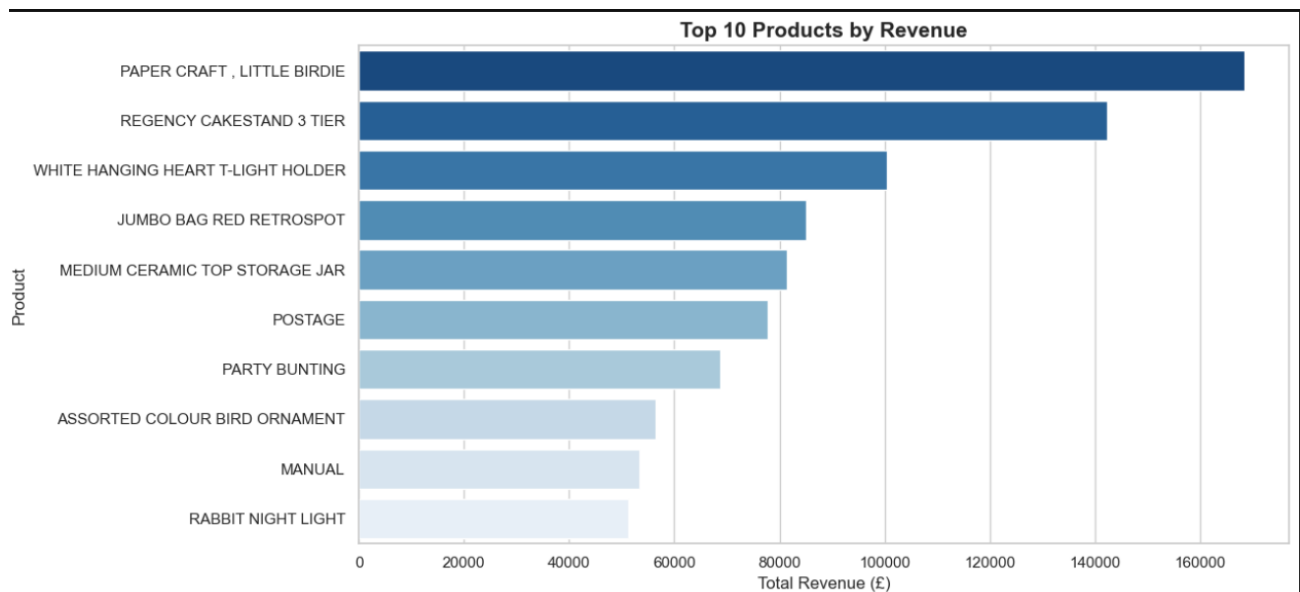
df[['Quantity', 'UnitPrice', 'Revenue']].describe()

# Optional safeguard: Standardize string casing to avoid splitting duplicate products
df['Description'] = df['Description'].str.upper()

top_products = (
    df.groupby('Description')['Revenue']
      .sum()
      .sort_values(ascending=False)
      .head(10)
)

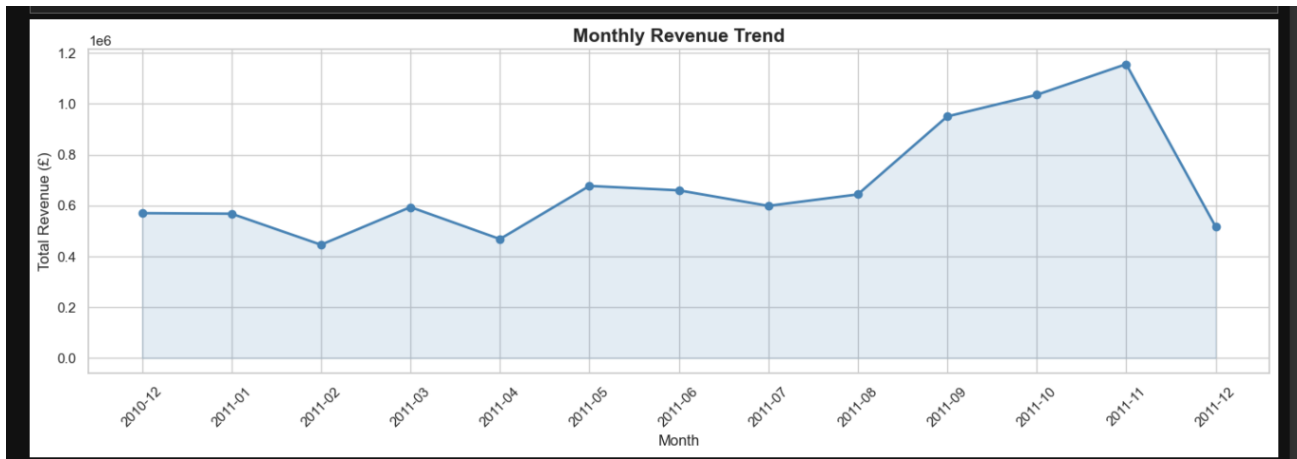
plt.figure(figsize=(13, 6))
sns.barplot(x=top_products.values, y=top_products.index, palette='Blues_r')
plt.title('Top 10 Products by Revenue', fontsize=15, fontweight='bold')
plt.xlabel('Total Revenue (£)')
plt.ylabel('Product')
plt.tight_layout()
plt.show()

```



```
monthly = (
    df.groupby(['Year','Month'])['Revenue']
      .sum()
      .reset_index()
)
monthly['Period'] = monthly['Year'].astype(str) + '-' + monthly['Month'].astype(str).str.zfill(2)
monthly.sort_values('Period', inplace=True)
```

```
plt.figure(figsize=(14, 5))
plt.plot(monthly['Period'], monthly['Revenue'], marker='o', color='steelblue', linewidth=2)
plt.fill_between(monthly['Period'], monthly['Revenue'], alpha=0.15, color='steelblue')
plt.title('Monthly Revenue Trend', fontsize=15, fontweight='bold')
plt.xlabel('Month')
plt.ylabel('Total Revenue (£)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
regional = (
    df.groupby('Country')['Revenue']
    .sum()
    .sort_values(ascending=False)
    .head(10)
)
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

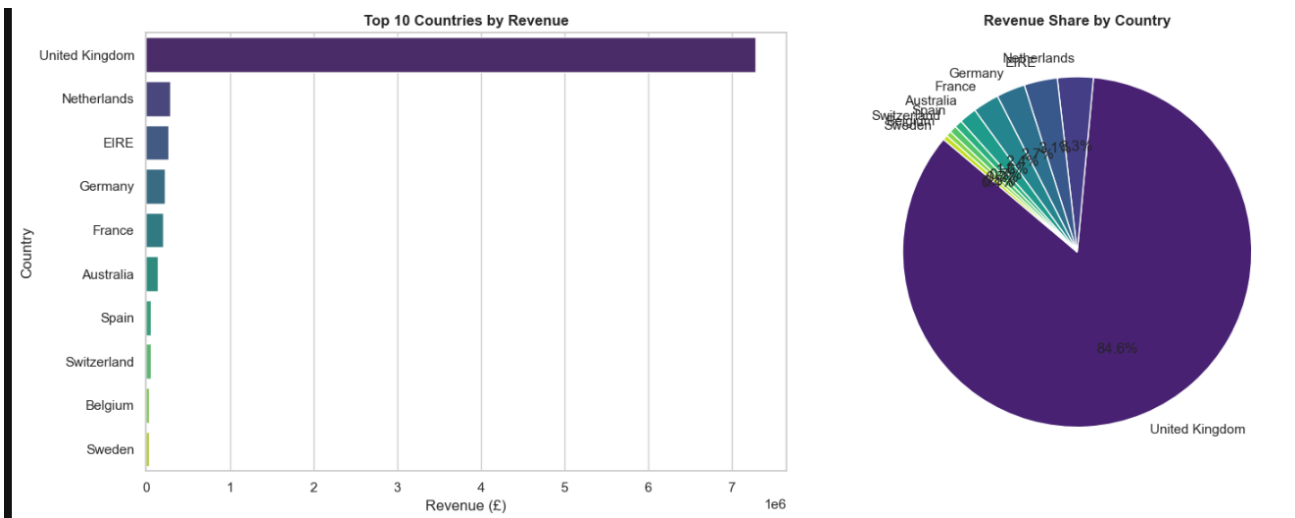
```
# Bar chart
```

```
sns.barplot(x=regional.values, y=regional.index, palette='viridis', ax=axes[0])
axes[0].set_title('Top 10 Countries by Revenue', fontweight='bold')
axes[0].set_xlabel('Revenue (£)')
```

```
# Pie chart
```

```
axes[1].pie(regional.values, labels=regional.index, autopct='%1.1f%%',
            startangle=140, colors=sns.color_palette('viridis', len(regional)))
axes[1].set_title('Revenue Share by Country', fontweight='bold')
```

```
plt.tight_layout()
plt.show()
```



```
corr = df[['Quantity','UnitPrice','Revenue']].corr()
```

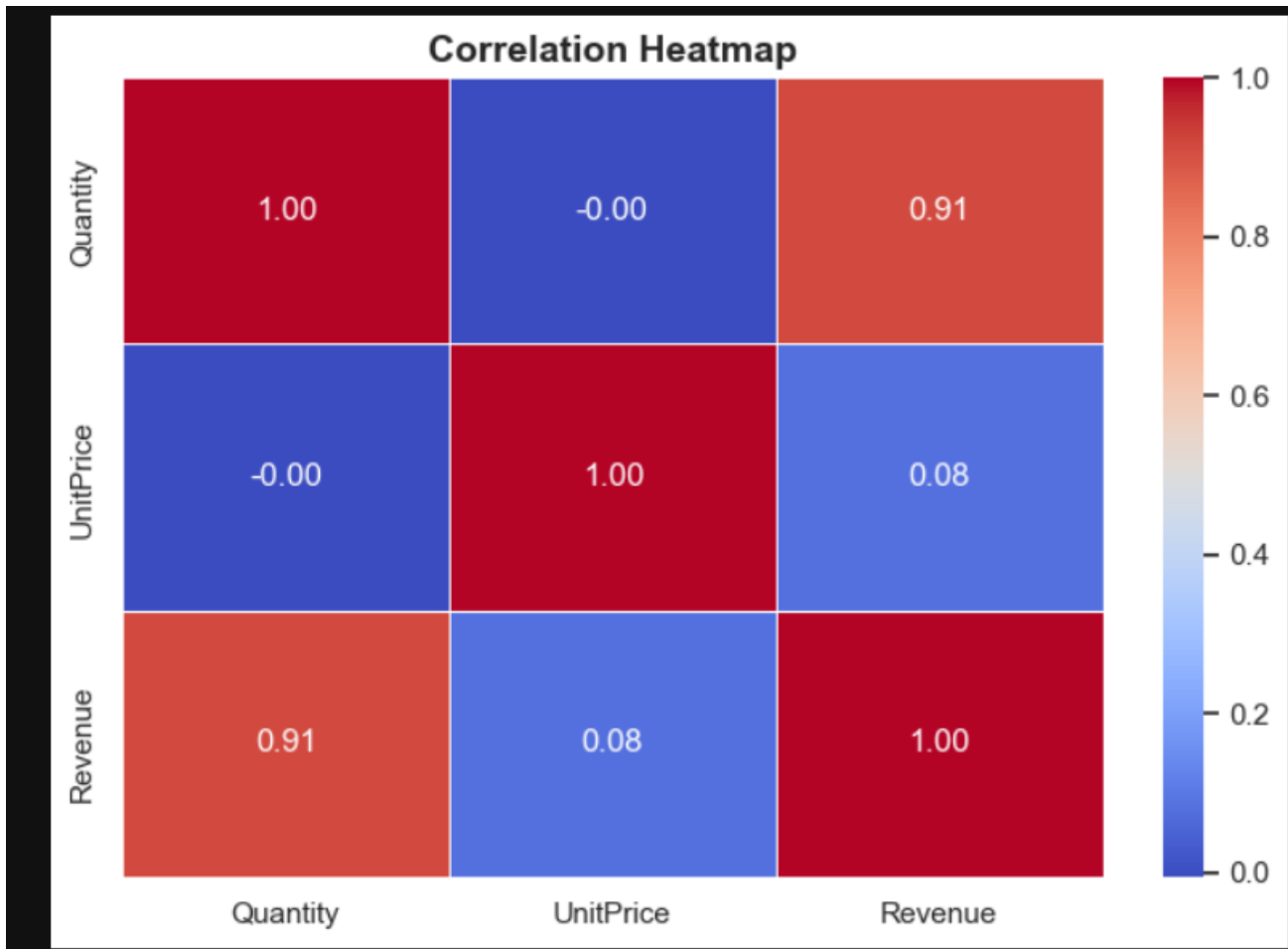
```
plt.figure(figsize=(7, 5))
```

```
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5)
```

```
plt.title('Correlation Heatmap', fontsize=14, fontweight='bold')
```

```
plt.tight_layout()
```

```
plt.show()
```



```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Cap at 99th percentile for visibility
```

```
rev_cap = df['Revenue'].clip(upper=df['Revenue'].quantile(0.99))
```

```
qty_cap = df['Quantity'].clip(upper=df['Quantity'].quantile(0.99))
```

```
sns.boxplot(y=rev_cap, ax=axes[0], color='lightcoral')
```

```
axes[0].set_title('Revenue Distribution (Box Plot)', fontweight='bold')
```

```
axes[0].set_ylabel('Revenue (£)')
```

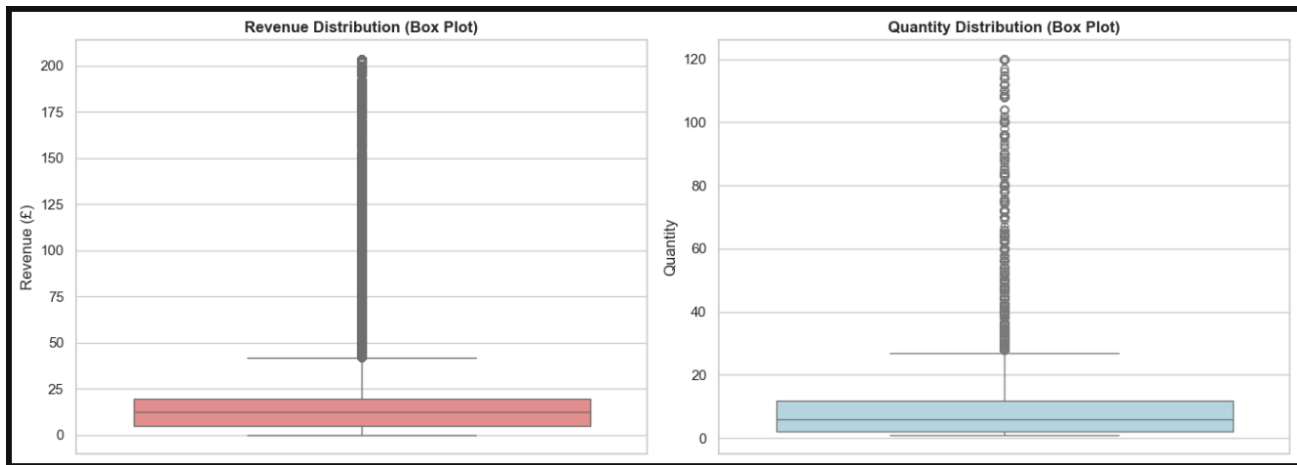
```
sns.boxplot(y=qty_cap, ax=axes[1], color='lightblue')
```

```
axes[1].set_title('Quantity Distribution (Box Plot)', fontweight='bold')
```

```
axes[1].set_ylabel('Quantity')
```

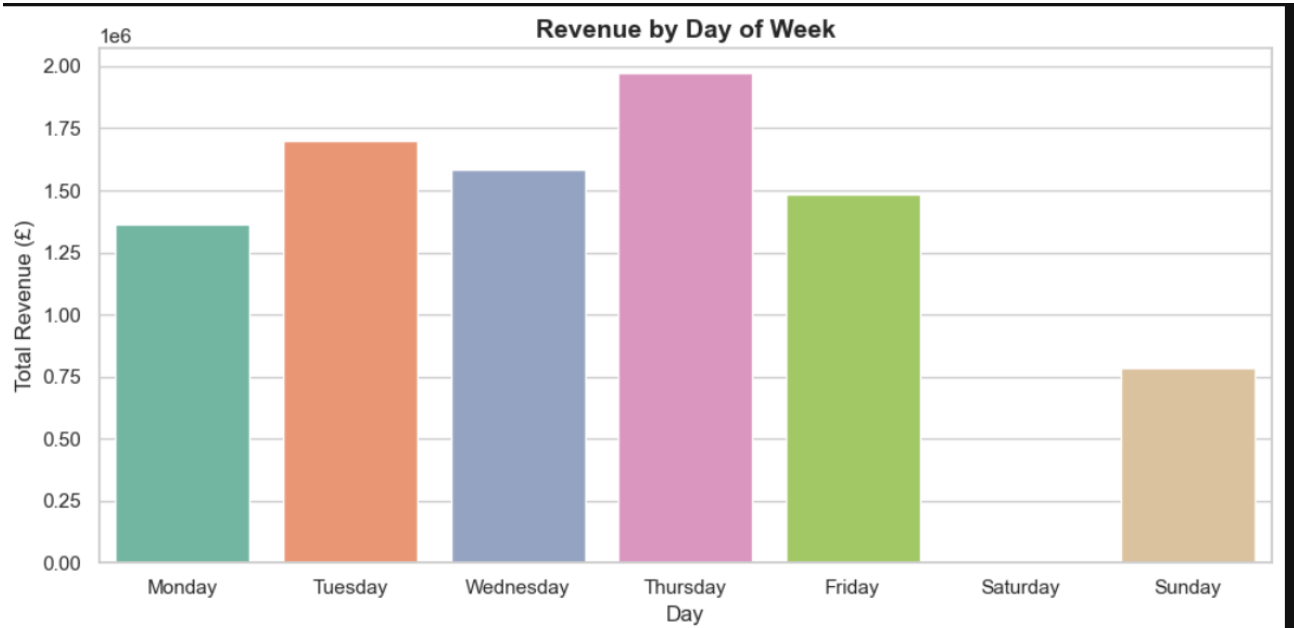
```
plt.tight_layout()
```

```
plt.show()
```



```
day_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
day_sales = df.groupby('DayOfWeek')['Revenue'].sum().reindex(day_order)
```

```
plt.figure(figsize=(10, 5))
sns.barplot(x=day_sales.index, y=day_sales.values, palette='Set2')
plt.title('Revenue by Day of Week', fontsize=14, fontweight='bold')
plt.xlabel('Day')
plt.ylabel('Total Revenue (£)')
plt.tight_layout()
plt.show()
```

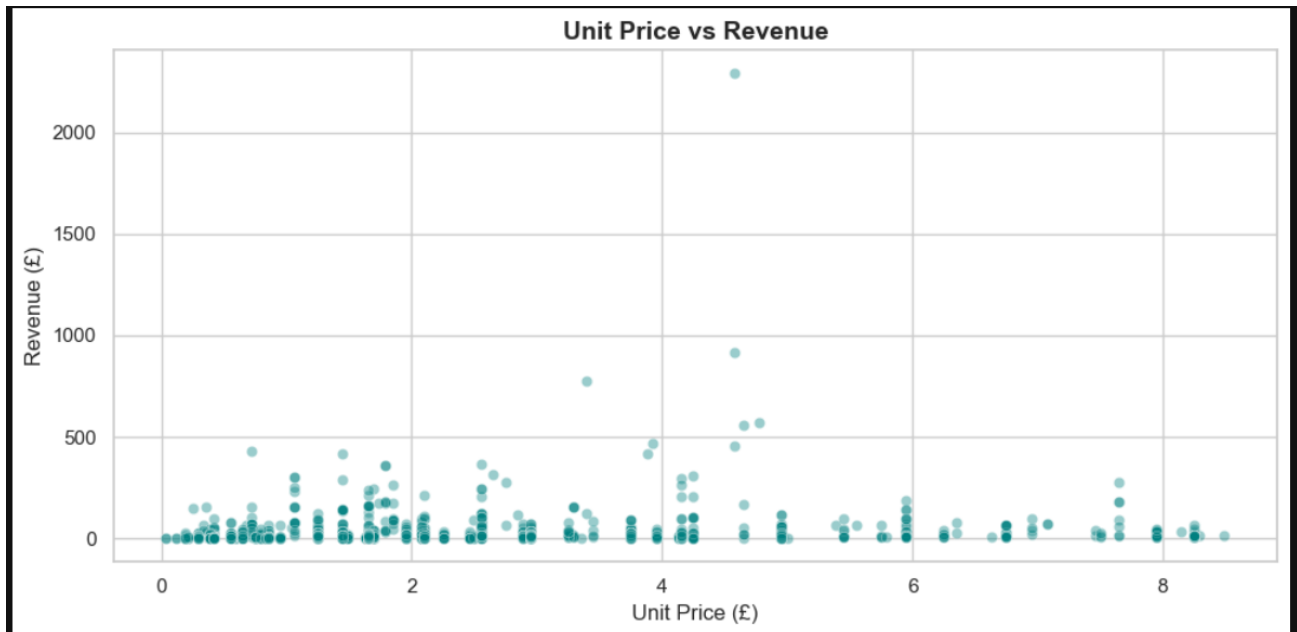


```
sample = df[df['UnitPrice'] < df['UnitPrice'].quantile(0.95)].sample(3000, random_state=42)
```

```

plt.figure(figsize=(10, 5))
sns.scatterplot(data=sample, x='UnitPrice', y='Revenue', alpha=0.4, color='teal')
plt.title('Unit Price vs Revenue', fontsize=14, fontweight='bold')
plt.xlabel('Unit Price (£)')
plt.ylabel('Revenue (£)')
plt.tight_layout()
plt.show()

```



```

# — Calculate RFM —
snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

rfm = df.groupby('CustomerID').agg(
    Recency = ('InvoiceDate', lambda x: (snapshot_date - x.max()).days),
    Frequency = ('InvoiceNo', 'nunique'),
    Monetary = ('Revenue', 'sum')
).reset_index()

print('RFM Table (first 5 rows):')
rfm.head()

# — Score RFM (1–4 scale) —
rfm['R_Score'] = pd.qcut(rfm['Recency'], q=4, labels=[4,3,2,1]).astype(int)
rfm['F_Score'] = pd.qcut(rfm['Frequency'].rank(method='first'), q=4, labels=[1,2,3,4]).astype(int)

```

```
rfm['M_Score'] = pd.qcut(rfm['Monetary'].rank(method='first'), q=4, labels=[1,2,3,4]).astype(int)
```

```
rfm['RFM_Score'] = rfm['R_Score'] + rfm['F_Score'] + rfm['M_Score']
```

```
# — Segment Customers —
```

```
def segment(score):
```

```
    if score >= 10:
```

```
        return 'Champions'
```

```
    elif score >= 7:
```

```
        return 'Loyal'
```

```
    elif score >= 5:
```

```
        return 'At-Risk'
```

```
    else:
```

```
        return 'Lost'
```

```
rfm['Segment'] = rfm['RFM_Score'].apply(segment)
```

```
print(rfm['Segment'].value_counts())
```

```
rfm.head()
```

```
# — Segment Count Plot —
```

```
plt.figure(figsize=(9, 5))
```

```
order = ['Champions', 'Loyal', 'At-Risk', 'Lost']
```

```
sns.countplot(data=rfm, x='Segment', order=order,
```

```
              palette=['#2ecc71', '#3498db', '#e67e22', '#e74c3c'])
```

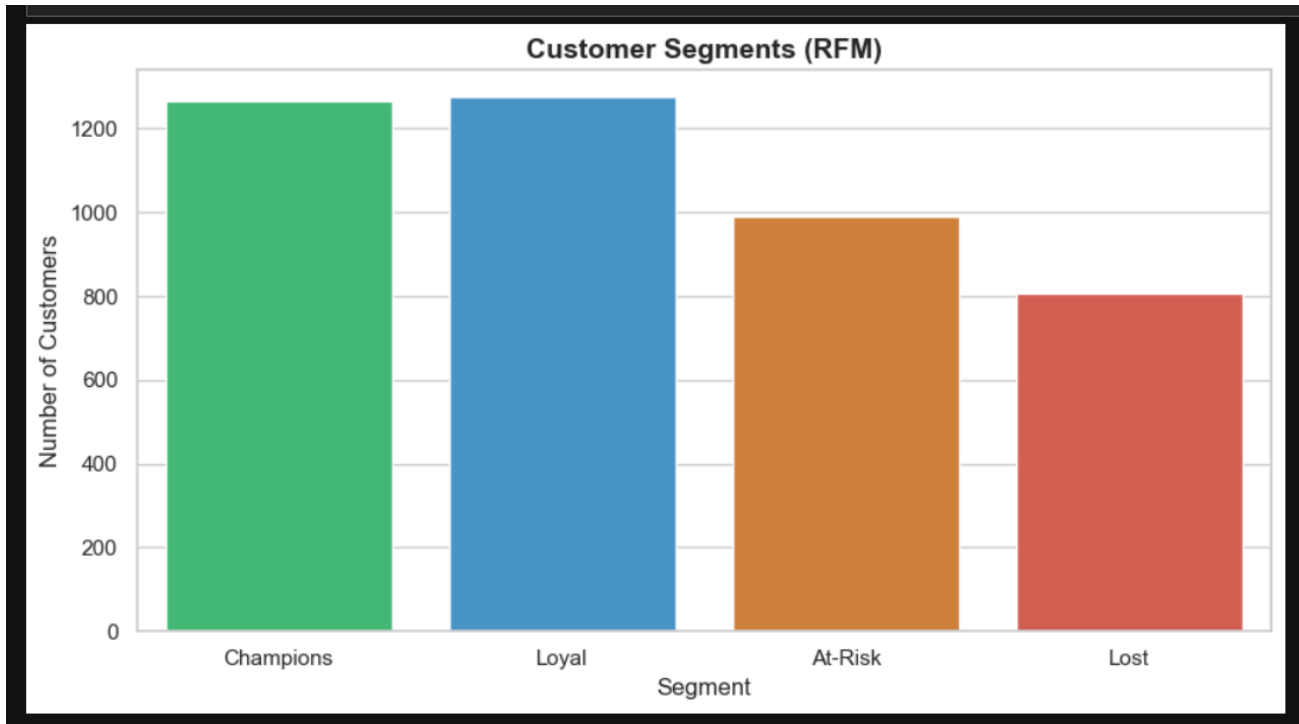
```
plt.title('Customer Segments (RFM)', fontsize=14, fontweight='bold')
```

```
plt.xlabel('Segment')
```

```
plt.ylabel('Number of Customers')
```

```
plt.tight_layout()
```

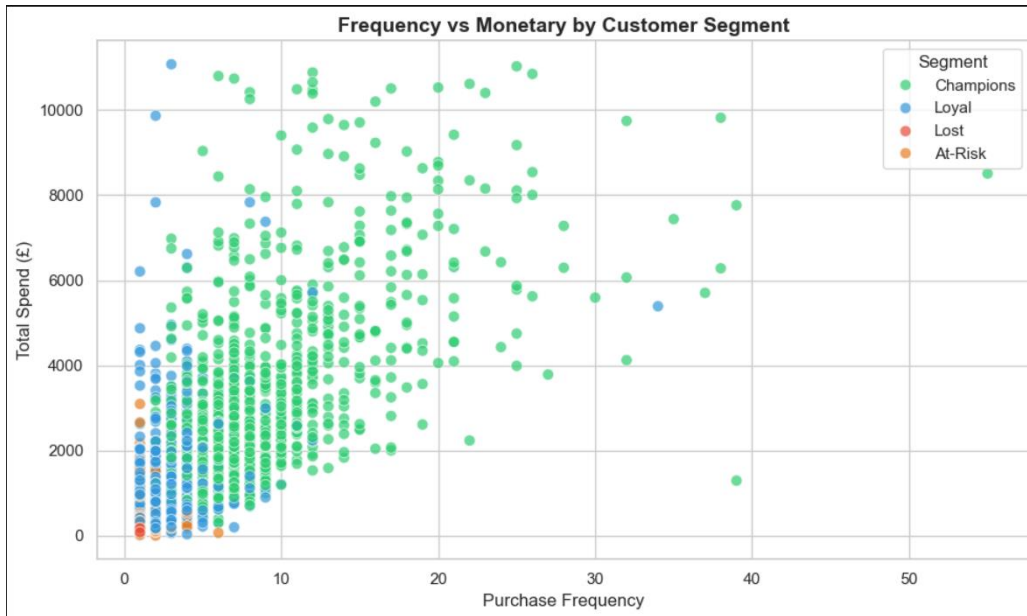
```
plt.show()
```



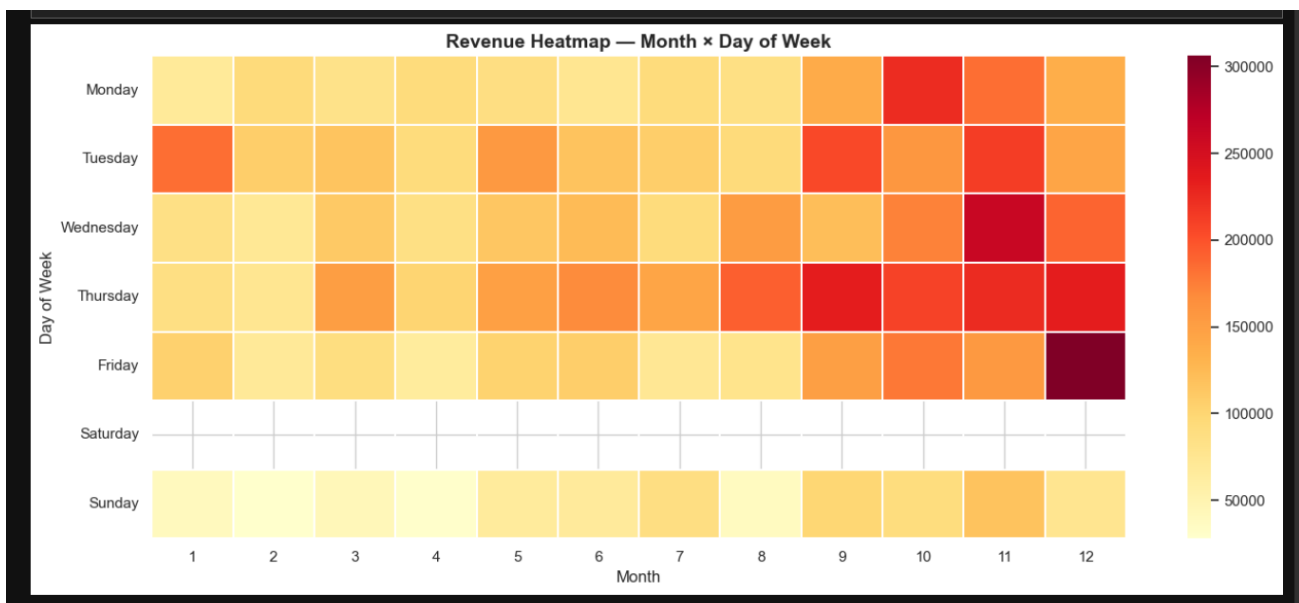
— Monetary vs Frequency scatter by Segment —

```
plt.figure(figsize=(10, 6))
palette = {'Champions': '#2ecc71', 'Loyal': '#3498db', 'At-Risk': '#e67e22', 'Lost': '#e74c3c'}
sample_rfm = rfm[rfm['Monetary'] < rfm['Monetary'].quantile(0.98)]
sns.scatterplot(data=sample_rfm, x='Frequency', y='Monetary',
                hue='Segment', palette=palette, alpha=0.7, s=60)
plt.title('Frequency vs Monetary by Customer Segment', fontsize=14, fontweight='bold')
plt.xlabel('Purchase Frequency')
plt.ylabel('Total Spend (£)')
plt.legend(title='Segment')
plt.tight_layout()
plt.show()
```

— Monetary vs Frequency scatter by Segment —



```
pivot = df.pivot_table(values='Revenue', index='DayOfWeek', columns='Month',
                        aggfunc='sum').reindex(day_order)
plt.figure(figsize=(14, 6))
sns.heatmap(pivot, cmap='YlOrRd', linewidths=0.3, fmt='.0f', annot=False)
plt.title('Revenue Heatmap — Month × Day of Week', fontsize=14, fontweight='bold')
plt.xlabel('Month')
plt.ylabel('Day of Week')
plt.tight_layout()
plt.show()
```



```

print('='*55)
print('      BUSINESS INSIGHTS SUMMARY')
print('='*55)
print(f"Total Transactions : {len(df):,}")
print(f"Total Revenue      : £{df['Revenue'].sum():,.2f}")
print(f"Unique Customers    : {df['CustomerID'].nunique():,}")
print(f"Unique Products     : {df['Description'].nunique():,}")
print(f"Countries Served    : {df['Country'].nunique():,}")
print(f"Top Country          : {df.groupby('Country')['Revenue'].sum().idxmax()}")
print(f"Best Month (Revenue) : {monthly.loc[monthly['Revenue'].idxmax(), 'Period']}")
print(f"Avg Order Value      : £{df['Revenue'].mean():.2f}")
print('='*55)
print('RFM Segments:')
for seg, cnt in rfm['Segment'].value_counts().items():
    print(f" {seg:<15}: {cnt} customers")
print('='*55)

```

Business_Summary_Output:

```

=====
                BUSINESS INSIGHTS SUMMARY
=====
Total Transactions : 392,692
Total Revenue     : £8,887,208.89
Unique Customers  : 4,338
Unique Products   : 3,877
Countries Served  : 37
Top Country       : United Kingdom
Best Month (Revenue) : 2011-11
Avg Order Value   : £22.63
-----
RFM Segments:
  Loyal      : 1276 customers
  Champions  : 1267 customers
  At-Risk    : 989 customers
  Lost       : 806 customers
=====

```

9. Timeline

Day	Task	Deliverable
Day 1	Data collection and environment setup	Raw dataset + Jupyter environment ready
Day 2	Data cleaning and preprocessing	Cleaned and preprocessed CSV file
Day 3	Descriptive statistics & feature engineering	Statistical summary + NumPy analysis
Day 4	EDA — trends, products, regions	EDA charts + bar/line/pie visualizations
Day 5	RFM Analysis + customer segmentation	Segmentation plots + RFM score table
Day 6	Advanced Seaborn visualizations + heatmaps	10+ complete visualization plots
Day 7	Final report, insights & submission	Complete Jupyter Notebook + final report

10. References

[1] Wes McKinney, Python for Data Analysis, 3rd Edition, O'Reilly Media, 2022.

- [2] Jake VanderPlas, Python Data Science Handbook, O'Reilly Media, 2023.
- [3] YT: CampusX
- [4] Kaggle E-Commerce Dataset: <https://www.kaggle.com/datasets/carrie1/ecommerce-data>
- [5] Seaborn Documentation: <https://seaborn.pydata.org>
- [6] Matplotlib Documentation: <https://matplotlib.org>
- [7] Pandas Documentation: <https://pandas.pydata.org>
- [8] NumPy Documentation: <https://numpy.org/doc>

Conclusion

🔍 Key Business Insights & Conclusions

1. **Top Product:** *PAPER CRAFT, LITTLE BIRDIE* is the highest revenue-generating product, significantly ahead of others — strong candidate for promotional focus.
2. **Seasonal Peak:** Revenue spikes sharply in **October–November 2011**, likely due to Christmas holiday shopping. Business should stock up inventory by September.
3. **UK Dominance:** United Kingdom contributes over **80% of total revenue**. International markets (Netherlands, EIRE, Germany) are present but minimal — opportunity for global expansion.
4. **Thursday is Best Day:** Thursday generates the highest weekday revenue. Marketing campaigns and promotions should be timed mid-week (Tue–Thu) for maximum impact.
5. **Weekend Drop:** Saturday and Sunday show near-zero revenue — confirming this is a **B2B wholesale retailer**. Consumer-facing businesses wouldn't see this pattern.
6. **RFM Segmentation:** ~1267 Champions and ~1276 Loyal customers form the core revenue base. The ~806 Lost customers should be targeted with re-engagement campaigns.
7. **Quantity drives Revenue:** Correlation of 0.91 between Quantity and Revenue shows volume-based selling is the primary revenue driver, not high unit prices.